

Sistemas Operacionais 2 - Processos e Threads

Prof. Rafael Milbradt

Contato: rmilbradt@gmail.com

Ementa

- UNIDADE 2 – GERÊNCIA DE PROCESSOS
 - 2.1 – Conceituação
 - 2.2 – Contextos de Software e Hardware
 - 2.3 – Estados do Processo e Mudanças de estados
 - 2.4 – Tipos de Processos
 - 2.5 – Threads - conceituação
 - 2.6 – Threads - implementação

19/03/2026

Prof. Rafael

2

Processos

- Processo: um programa ou aplicativo em execução;
- Gerência dos processos é uma das principais funções do SO:
 - Alocação e compartilhamento de recursos de E/S e dados;
 - Sincronismo para controle de concorrência;
 - Compartilhamento do processador;

19/03/2026

Prof. Rafael

3

Processos

- Do ponto de vista da UCP instruções não tem ligação alguma com programas ou aplicativos.
 - Gerenciador de Processos implementa a multiprogramação efetivamente;
 - Conceito de Processo é necessário para implementar a alternância de uso da UCP;

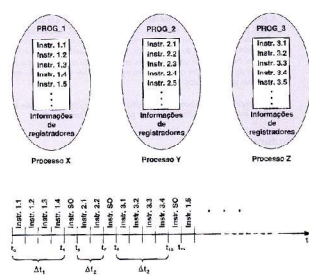
19/03/2026

Prof. Rafael

4

Processos

- Do ponto de vista da UCP, não existem processos nem SO.
- É o SO que controla a alternância entre os processos, tirando o processador de um e dando ao outro através das **Mudanças de Contexto**.



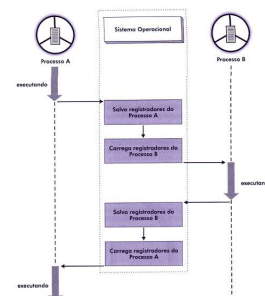
19/03/2026

Prof. Rafael

5

Processos

- Troca de Contexto: procedimento realizado pelo SO para alternar a execução entre um processo e outro.
- Salva o conteúdo dos registradores do processo que estava executando no espaço da MP designado ao processo.
- Escolhe o próximo processo a executar e restabelece o contexto de registradores salvo naquele processo.



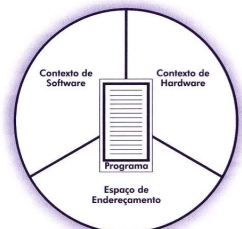
19/03/2026

Prof. Rafael

6

Processos

- O contexto de um processo consiste em todas as informações que descrevem um processo e são controladas pelo SO.
- Dividem-se da forma:



19/03/2026

Prof. Rafael

7

Processos

- Contexto de Hardware:**
 - Consiste no conteúdo dos registradores da UCP:
 - PC – Program Counter: endereço da próxima instrução a ser executada;
 - SP – Stack Pointer: endereço do topo da pilha;
 - PSW – Program Status Word: registrador de status do processador;
 - RI – Registrador de Instrução: não é necessário, pois a instrução atual vai ser concluída antes da troca de contexto.
 - Registradores gerais: dados em uso pelo processo;

19/03/2026

Prof. Rafael

8

Processos

- Contexto de Software:**
 - Prioridade de execução;
 - Arquivos abertos;
 - Recursos de E/S alocados;
 - Identificação:
 - PID: process ID;
 - UID: user ID;
 - Quotas:
 - Limites máximos de arquivos abertos, operações de E/S, tamanho máximo dos buffers de E/S, subprocessos que podem ser criados;
 - Privilegios:
 - Direitos do processo em relação a ele mesmo e aos demais;
 - Por exemplo: processo pode aumentar a prioridade ou pode alterar suas quotas?

19/03/2026

Prof. Rafael

9

Processos

- Espaço de Endereçamento:**
 - Definição da área de MP reservada ao processo;
 - Maiores detalhes na Unidade sobre Gerência de Memória;



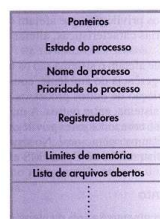
19/03/2026

Prof. Rafael

10

Processos

- Bloco de Controle do Processo - Process Control Block (PCB):**
 - Estrutura de dados para descrição de cada processo;
 - Contém os três contextos já citados;
 - Ocupa espaço na MP, logo existe restrição ao número de processos no SO;



19/03/2026

Prof. Rafael

11

Processos

- Estados do Processo:**
 - Num sistema multiprogramável, nenhum processo ocupa exclusivamente a UCP de forma que todos se alternam em diferentes estados ao longo de sua vida no SO.
- Estados Possíveis:**
 - Execução (running): processo em execução na UCP.
 - Pronto (ready): processo não está em execução, mas está pronto para executar quando o SO decidir por isto;
 - Espera (wait): processo não está executando nem está pronto. Está esperando por algum recurso, operação de E/S ou mesmo a uma data/hora programada.
- Listas de processo:**
 - SO organiza os processos em listas encadeadas de acordo com o estado para facilitar operações de escalonamento e alterações de estado.

19/03/2026

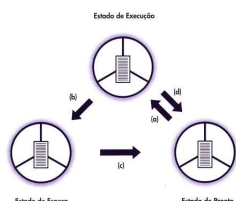
Prof. Rafael

12

Processos

Mudanças de Estado:

- **Pronto → Execução**: um processo em estado de pronto pode ganhar a CPU conforme a política de escalonamento da UCP.
- **Execução → Espera**: processo executa operação de E/S ou tenta alocar algum recurso não disponível.
- **Espera → Pronto**: operação que havia o colocado em estado de espera é atendida.
- **Execução → Pronto**: término da fatia de tempo de UCP daquele processo ou entrada de outro processo com maior prioridade de execução bem como outras decisões que são políticas do escalonador.



19/03/2026

Prof. Rafael

13

Processos

Criação e Eliminação de Processos:

- Além dos estados já descritos existem outros dois relacionados à criação e eliminação de processos:
- **Criação (new)**: SO já criou o PCB porém por alguma política sua não o colocou em estado de pronto.
- **Terminado (exit)**: processo já encerrou e não está mais ativo. O PCB ainda existe para o SO desalocar recursos e contabilizar estatísticas sobre o processo.

19/03/2026

Prof. Rafael

14

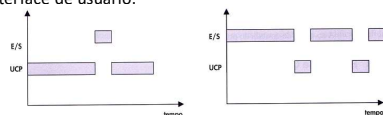
13

14

Processos

Classificações de Processos:

- De acordo com o comportamento dos processos no uso de recursos como UCP e E/S eles classificam-se em:
 - **CPU-bound (ligado à UCP)**: praticamente não realizam operações de E/S, alternando-se quase todo tempo entre os estados Pronto e Execução. Ex.: aplicações científicas.
 - **I/O Bound (ligado à E/S)**: realizam muitas operações de E/S, permanecendo pouco tempo no estado de Execução. Ex.: aplicações comerciais, aplicações de rede, aplicações com interface de usuário.



19/03/2026

Prof. Rafael

15

15

Processos

Foreground e Background:

- Outra classificação do comportamento dos processos em relação ao usuário;
- **Foreground**: aguardam comando do usuário (entrada), realizam alguma operação e retornam uma resposta ao usuário (saída);
- **Background**: a entrada bem como a saída podem ser por meio de arquivo, conexão de rede, etc. Não realizam operações diretamente com o usuário.
 - É possível a saída de um processo ser usada como entrada de outro com o uso de um mecanismo conhecido como "pipe".

19/03/2026

Prof. Rafael

16

16

Exercícios de Fixação

1. Por que o conceito de processo é tão importante no projeto de sistemas multiprogramáveis?
2. O que é o contexto de hardware e como é a implementação da troca de contexto?
3. Dê um exemplo que apresente todas as mudanças de estado de um processo, juntamente com o evento gerador.
4. Dê exemplos de aplicações CPU-Bound e I/O-Bound.

19/03/2026

Prof. Rafael

17

17

Processos

Formas de Criação de Processos:

- Logon interativo:
 - Processo criado para atender a comandos do usuário após o logon no SO.
- Via Linguagem de Comandos:
 - O usuário pode usar o terminal criado no logon para criar outros processos em foreground ou background através da linguagem de comandos.
- Via System Call:
 - Através de chamada de sistema dentro de um aplicativo de usuário;

19/03/2026

Prof. Rafael

18

18

Processos

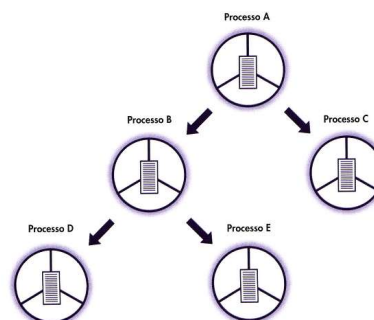
- Processos Independentes, Subprocessos e Threads:
 - Formas de implementar concorrência e paralelismo dentro dos programas;
 - Para programa poder fazer E/S e manter a interface atualizada, precisará de paralelismo;
 - **Subprocesso**: processo que possui um processo pai associado. Quando o pai é terminado os filhos automaticamente são terminados;
 - Subprocessos compartilham quotas com o processo pai.
 - Cada um possui sua própria PCB;

19/03/2026

Prof. Rafael

19

Processos



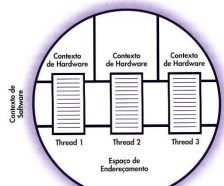
19/03/2026

Prof. Rafael

20

Processos

- Thread:
 - Conceito criado especificamente para implementação de aplicações concorrentes;
 - É também conhecida como “processo-leve”;



19/03/2026

Prof. Rafael

21

Processos

- Sinais:
 - Mecanismo de comunicação entre processos bem como do SO para com os processos;



- Exceções em tempo de execução geram sinais para os processos;

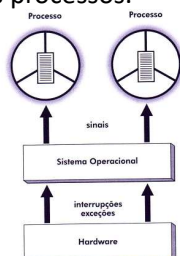
19/03/2026

Prof. Rafael

22

Processos

- Interrupções de hardware também geram sinais para os processos.



19/03/2026

Prof. Rafael

23

Threads

- São processos leves utilizados exclusivamente para implementar concorrência dentro de aplicações;
- Ambiente Monothread:
 - Aplicações concorrentes precisa ser implementadas através de subprocessos:
 - Maior consumo de recursos do sistema;
 - Tempo de processamento para criar e destruir processos;
 - Comunicação entre processos é feita pelo SO através de pipes, troca de mensagens e outros métodos;
 - Compartilhamento entre recursos como arquivos abertos, memória é muito difícil;

19/03/2026

Prof. Rafael

24

Threads

- Ambiente Multithread:
 - Programas dentro de um mesmo aplicativo são associados às threads e não a subprocessos;
 - Um processo pode ter várias threads que compartilham o mesmo espaço de endereçamento;
 - Comunicação entre as threads é feita através da MP, sem a necessidade de interferência do SO;
 - Threads compartilham o processador da mesma forma que processos e passam pelas mesmas mudanças de estado (execução, espera e pronto)
 - Na verdade nos SOs modernos multithread, apenas as threads concorrem pelo processador;
 - Processos tem no mínimo uma thread;
 - Uma thread pode fazer E/S, enquanto outra processa;

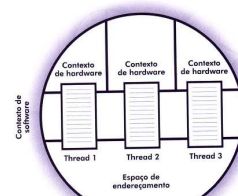
19/03/2026

Prof. Rafael

25

Threads

- Threads armazenam apenas o contexto de hardware, já o contexto de software é compartilhado por todo o processo;



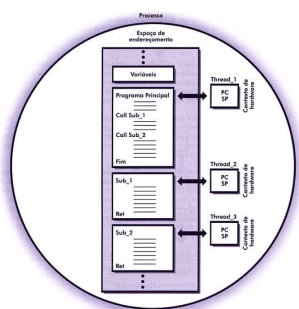
19/03/2026

Prof. Rafael

26

Threads

- Contexto de HW → TCB (Thread Control Block)
- Ambientes Monothread:
 - Unidade de alocação de Recursos → Processo;
 - Unidade de Escalonamento → Processo;
- Ambientes Multithread:
 - Unidade de alocação de Recursos → Processo;
 - Unidade de Escalonamento → Thread;



19/03/2026

Prof. Rafael

27

Threads

- Threads compartilham espaço de endereçamento e recursos sem interferência do SO com todas as vantagens e desvantagens:
 - Rapidez e simplicidade;
 - É necessário controlar a concorrência pelos recursos;

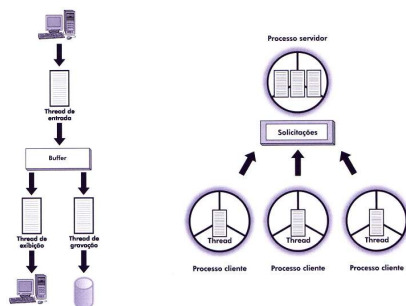
19/03/2026

Prof. Rafael

28

Threads

- Programação Multithread:



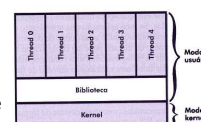
19/03/2026

Prof. Rafael

29

Threads

- Implementação das Threads:
 - TMU – Thread em Modo Usuário:
 - São implementadas pela aplicação e não pelo SO;
 - Existe uma biblioteca dentro da aplicação que implementa as threads e o escalonamento;
 - SO não sabe da existência de múltiplas threads;
 - Se uma thread faz uma chamada de sistema bloqueante, todo o processo fica em estado de espera mesmo que existam threads em estado de pronto;
 - Tratamento de interrupções:
 - Processo precisa identificar de que thread a interrupção pertence;
 - Processo precisa interceptar a interrupção de clock para implementar o escalonamento.
 - Múltiplos processadores: não é possível ter multitarefa real;



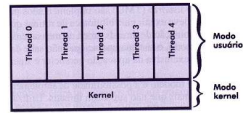
19/03/2026

Prof. Rafael

30

Threads

- TMK – Thread em Modo Kernel:
 - SO tem conhecimento das threads;
 - Os problemas da implementação TMU não existem;
 - O grande problema é o baixo desempenho;
 - Mudanças frequentes no modo de acesso (usuário-kernel-usuário) causam a deterioração no desempenho;



19/03/2026

Prof. Rafael

31

Threads

Tabela 6.3 Comparação entre tempos de latência (Anderson et al., 1992)

Implementação	Operação 1 (μs)	Operação 2 (μs)
Subprocessos	11.300	1.840
Threads em Modo Kernel	948	441
Threads em Modo Usuário	34	37

- Operação 1: criação, escalonamento e execução;
- Operação 2: eliminação;

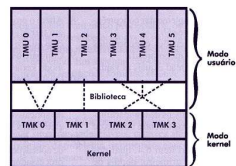
19/03/2026

Prof. Rafael

32

Threads

- TMH – Threads em Modo Híbrido:
 - Misto dos dois mundos;
 - Um nível de threads em TMU e outro nível em TMK;
 - Programador desenvolve em termos de TMU e especifica quantas TMKs serão associadas ao processo;
 - Vantagens e desvantagens herdadas dos dois modos;



19/03/2026

Prof. Rafael

33

Fim

- Fonte para conteúdo e ilustrações:
 - MACHADO, Francis Berenger; MAIA, Luiz Paulo. **Arquitetura de Sistemas Operacionais**. Editora LTC, 5ª Ed., 2013.

19/03/2026

Prof. Rafael

34